

**UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET**

SEMINARSKI RAD
IZ PREDMETA PRETRAŽIVANJE INFORMACIJA

SISTEM ZA PRETRAŽIVANJE POSLOVA
Bulov i vektorski model nad polustrukturiranim oglasima za posao

Mentor: Prof. dr Suzana Stojković

Student: Pavle Milošević
Broj indeksa: 19244

Niš, 2026. godine

Contents

1. Uvod	3
2. Teorijske osnove pretraživanja informacija.....	3
2.1. Invertovani indeks i Bulov model.....	3
2.2. Vektorski model, TF-IDF i kosinusna sličnost.....	3
3. Korpus podataka i organizacija projekta	4
4. Obrada teksta	4
5. Izgradnja invertovanog indeksa	5
5.1. Prednosti izabrane strukture	5
5.2. Veza sa obradom teksta	5
6. Modeli pretrage.....	6
6.1. Bulov model.....	6
6.2. Vektorski model	6
7. Polustrukturirani podaci i filtriranje	7
8. Korisnički interfejs	7
9. Rezultati i evaluacija sistema.....	9
10. Prednosti, ograničenja i moguća unapređenja.....	10
10.1. Ograničenja	10
10.2. Moguća unapređenja	10
11. Zaključak.....	11
12. Literatura	11

1. Uvod

Pretraživanje informacija predstavlja oblast koja se bavi organizovanjem, indeksiranjem i pronalaženjem dokumenata relevantnih za korisnički upit. U klasičnom IR sistemu dokumenti se najpre pripremaju i indeksiraju, a zatim se korisnički upit obrađuje istim ili kompatibilnim postupkom kako bi se pronašli dokumenti koji najbolje odgovaraju traženom sadržaju.

Cilj ovog projekta je razvoj sistema za pretraživanje oglasa za posao. Dokumenti nisu predstavljeni samo jednim slobodnim tekstom, već su polustrukturirani i sadrže odvojena polja: identifikator oglasa, naslov pozicije, kompaniju, lokaciju i opis posla. Takva struktura omogućava da se tekstualna pretraga kombinuje sa filtriranjem po metapodacima.

Sistem podržava dva modela pretrage. Bulov model koristi operatore AND, OR i NOT i vraća skup dokumenata koji zadovoljavaju zadate logičke uslove. Vektorski model koristi TF-IDF reprezentaciju i kosinusnu sličnost, pa rezultate rangira prema stepenu sličnosti sa upitom. Korisnički interfejs je realizovan u biblioteci Streamlit, dok se kvalitet oba modela proverava nad ručno označenim skupom relevantnih dokumenata.

Implementacija je organizovana kroz odvojene komponente za obradu teksta, izgradnju invertovanog indeksa, Bulovu pretragu, TF-IDF rangiranje i evaluaciju pomoću mera Precision, Recall i F1.



Slika 1: Logički tok obrade u razvijenom sistemu za pretraživanje poslova.

2. Teorijske osnove pretraživanja informacija

2.1. Invertovani indeks i Bulov model

Invertovani indeks za svaki termin čuva skup identifikatora dokumenata u kojima se termin pojavljuje. Umesto sekvencijalnog pregleda svih oglasa, pretraga direktno pristupa skupu konkretnog termina. Bulovi operatori se zatim realizuju skupovnim operacijama: AND presekom, OR unijom, a NOT razlikom skupova.

2.2. Vektorski model, TF-IDF i kosinusna sličnost

Vektorski model predstavlja dokument i upit kao vektore termina. TF-IDF dodeljuje veću težinu terminima koji su značajni u konkretnom dokumentu, a manje su česti u celoj kolekciji. Sličnost upita i oglasa računa se kosinusnom sličnošću. Što je ugao između vektora manji, vrednost sličnosti je veća i oglas se rangira više.

3. Korpus podataka i organizacija projekta

Korpus se nalazi u fajlu jobs.json i sastoji se od 1.000 sintetički generisanih IT oglasa. Svaki oglas predstavlja jedan dokument sa pet polja: id, naslov, kompanija, lokacija i opis. Generator koristi unapred definisane kategorije radnih mesta, različite nivoe senioriteta, kompanije, lokacije, tehnološke parametre i nekoliko šablona opisa, čime se dobija dovoljno raznovrstan skup za demonstraciju pretrage.

Za formalnu evaluaciju ne koristi se svih 1.000 oglasa, već kontrolisani podskup prvih 100 dokumenata. Za pet reprezentativnih upita ručno su označeni ID-jevi relevantnih oglasa i sačuvani u ground_truth.json. Time se rezultat sistema može uporediti sa unapred poznatim skupom relevantnih dokumenata.

Element	Uloga u projektu
app.py	Streamlit interfejs; izbor modela, polja, filtera i prikaz evaluacije
preprocessing.py	Normalizacija, tokenizacija, stop-reči, stemming i zaštita naziva tehnologija
stemmer.py	Postojeći stemmer za srpski jezik koji se poziva iz preprocessing modula
inverted_index.py	Izgradnja invertovanog indeksa i Bulova pretraga
vector_model.py	TF-IDF matrice, kosinusna sličnost i rangiranje rezultata
evaluation.py	Precision, Recall i F1 za Bulov i vektorski model
data/jobs.json	Glavni korpus oglasa
data/ground_truth.json	Ručno označeni relevantni dokumenti za test upite

Tabela 1: Glavni fajlovi i odgovornosti u projektu.

4. Obrada teksta

Isti postupak obrade teksta koristi se pri indeksiranju oglasa i pri obradi korisničkog upita. Time se obezbeđuje da isti pojam u dokumentu i upitu bude predstavljen na isti način. Tekst se prvo prevodi u mala slova, zatim se tokenizuje, uklanjaju se česte stop-reči i brojevi, a obične reči se prosleđuju stemmeru za srpski jezik.

Poseban problem kod IT oglasa predstavljaju nazivi tehnologija sa specijalnim znacima. Jednostavno uklanjanje interpunkcije bi termine C, C++ i C# svelo na isti oblik, što bi dovelo do netačnih rezultata. Zbog toga tokenizacija čuva znakove koji su deo naziva tehnologije, a skup zaštićenih termina obuhvata oblike kao što su c++, c#, .net, asp.net, node.js i vue.js. Zaštićeni termini se ne stemuju, dok se ostale reči obrađuju standardno.

Korak	Implementacija	Svrha
Mala slova	tekst.lower()	Ujednačava Python, PYTHON i python
Tokenizacija	regularni izraz prilagođen IT terminima	Čuva +, # i . kada su deo naziva tehnologije
Stop-reči	lista srpskih stop-reči	Uklanja česte reči male informacione vrednosti
Brojevi	isdigit()	Ne indeksira samostalne brojeve
Stemming	stem_arr	Svodi obične reči na zajednički koren
Zaštićene tehnologije	TEHNOLOGIJE skup	Sprečava da C++, C# i .NET budu pogrešno izmenjeni

Tabela 2: Pipeline obrade teksta.

5. Izgradnja invertovanog indeksa

Invertovani indeks se gradi funkcijom `kreiraj_indeks`. Pošto su naslov i opis tekstualna polja namenjena pretrazi, ona se obrađuju funkcijom `obrađi_tekst` i za svaki dobijeni termin se čuva skup ID-jeva oglasa u kojima se termin pojavljuje. Lokacija i kompanija se tretiraju kao strukturirani podaci i koriste se kao filteri, pa ih nije potrebno uključivati u tekstualni indeks.

Logička struktura indeksa može se prikazati na sledeći način:

```
indeks = {  
    "naslov": { termin: {id1, id2, ...} },  
    "opis": { termin: {id1, id2, ...} }  
}
```

Slika 2: Pojednostavljena struktura invertovanog indeksa.

Za svaki oglas najpre se dobijaju tokeni naslova i opisa. Ako termin još ne postoji u odgovarajućem delu indeksa, kreira se novi prazan skup, nakon čega se u njega dodaje ID trenutnog oglasa. Korišćenje skupa je pogodno jer isti ID ne može biti dodat više puta za isti termin, čak i ako se termin u dokumentu pojavljuje više puta.

Pronalaženje dokumenata za pojedinačni termin vrši funkcija `nadji_oglase_za_rec`. Kada korisnik izabere pretragu samo po naslovu ili samo po opisu, vraća se posting skup iz tog polja. Kada je izabrano „sve“, vrši se unija skupova naslova i opisa. Ovakvo razdvajanje polja čuva polustrukturiranu prirodu dokumenata i omogućava precizniju kontrolu pretrage.

5.1. Prednosti izabrane strukture

- pretraga termina ne zahteva prolazak kroz svih 1.000 oglasa;
- Bulovi operatori se direktno mapiraju na skupovne operacije u Pythonu;
- odvojeni indeksi za naslov i opis omogućavaju pretragu po konkretnom polju;
- filteri kompanije i lokacije ostaju jednostavni i nezavisni od tekstualnog indeksa.

5.2. Veza sa obradom teksta

Kvalitet invertovanog indeksa direktno zavisi od tokenizacije. Izmena kojom se čuvaju nazivi tehnologija naročito je važna upravo u ovom delu sistema: termini `c`, `c++` i `c#` sada dobijaju odvojene ključeve i mogu da vrate različite skupove oglasa. Time se sprečava situacija u kojoj upit „C“ vraća C# ili C++ oglas samo zato što je interpunkcija ranije bila uklonjena.

6. Modeli pretrage

6.1. Bulov model

Bulova pretraga podržava operatore AND, OR i NOT. Upit se najpre deli na OR grupe. Unutar svake grupe termini se podrazumevano kombinuju operatorom AND, odnosno presekom posting skupova. NOT postavlja negaciju za sledeći termin i taj skup se oduzima od trenutnog rezultata. Na kraju se rezultati svih OR grupa spajaju unijom.

Ovakav pristup automatski daje veći prioritet operatoru AND u odnosu na OR. Na primer, upit „Python OR Java AND Senior“ tretira se kao dve grupe: „Python“ i „Java AND Senior“, pa je rezultat Python OR (Java AND Senior). Istovremeno, upit „Python Backend“ radi kao implicitni AND, odnosno kao „Python AND Backend“.

podeli upit na OR grupe

za svaku grupu:

AND -> preskoči jer je podrazumevan

NOT -> zapamti negaciju sledećeg termina

termin -> pronađi posting skup

bez NOT: presek sa trenutnim rezultatom

sa NOT: oduzmi posting skup

spoji rezultate grupa unijom

Slika 3: Pseudokod pojednostavljenog Bulovog parsera.

Zagrade nisu podržane. Izraz (Python OR Java) AND Django može se zapisati ekvivalentno kao „Python AND Django OR Java AND Django“.

6.2. Vektorski model

Vektorski model koristi dve zasebne TF-IDF reprezentacije: jednu za naslove oglasa i jednu za opise. Za korisnički upit formiraju se odgovarajući TF-IDF vektori, a zatim se kosinusnom sličnošću porede sa svim oglasima. Kada se pretražuje celo tekstualno polje dokumenta, konačni skor se formira kao ponderisana kombinacija: 0,6 za naslov i 0,4 za opis. Time naslov dobija nešto veći značaj jer neposredno opisuje radnu poziciju.

Rezultati sa skorom većim od nule sortiraju se opadajuće, pa se korisniku prikazuju oglasi sa najvećom sličnošću. Za razliku od Bulovog modela, dokument ne mora strogo da zadovolji sve termine da bi bio prikazan; dovoljno je da ima pozitivnu TF-IDF sličnost sa upitom. Zbog toga vektorski model može vratiti i dokumente koji sadrže samo deo termina iz upita.

7. Polustrukturirani podaci i filtriranje

Važna osobina projekta je razdvajanje tekstualnih i strukturiranih atributa oglasa. Naslov i opis se pretražuju algoritmicima za pretraživanje informacija, dok kompanija i lokacija služe kao tačni filteri. Korisnik zato može najpre zadati tekstualni upit, a zatim suziti rezultate na konkretnu lokaciju ili kompaniju bez potrebe da se ti podaci tretiraju kao običan deo opisa.

Kod vektorskog modela najpre se dobija rangirana lista kandidata, zatim se primenjuju izabrani filteri i tek na kraju ograničenje maksimalnog broja prikazanih rezultata. Time se zadržavaju i validni filtrirani rezultati koji se mogu nalaziti niže na početnoj rang-listi. Kod Bulovog modela filteri se primenjuju direktno nad skupom pronađenih ID-jeva.

8. Korisnički interfejs

Korisnički interfejs je realizovan pomoću biblioteke Streamlit. Glavna kartica za pretragu sadrži polje za unos upita, izbor između vektorskog i Bulovog modela, izbor tekstualnog polja „sve“, „naslov“ ili „opis“, klizač za maksimalan broj rezultata i dva strukturirana filtera: lokaciju i kompaniju.

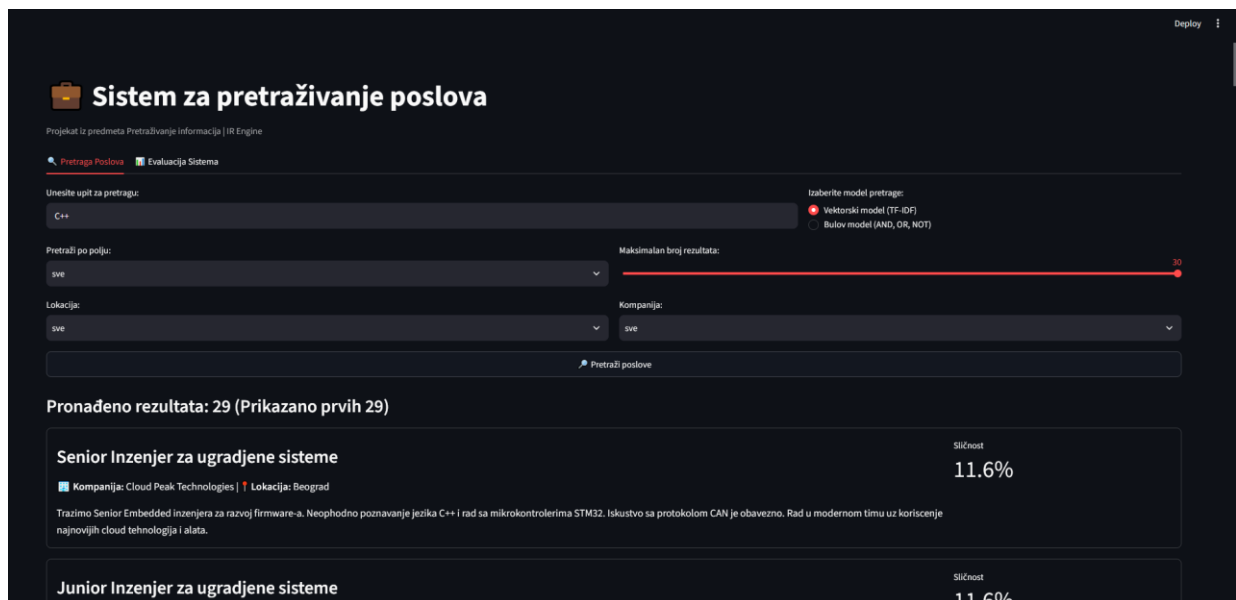
Oba modela rezultate pripremaju u zajedničkom obliku koji sadrži oglas i opcioni skor. Zbog toga se prikaz rezultata može realizovati jednom petljom. Za vektorski model prikazuje se procenat sličnosti, dok Bulov model prikazuje identifikator oglasa jer njegov rezultat nije rangiran numeričkim skorom.

Druga kartica interfejsa namenjena je evaluaciji. Jednim dugmetom pokreće se testiranje nad ručno označenim ground truth skupom, nakon čega se prikazuje tabela Precision, Recall i F1 vrednosti za svih pet upita, kao i prosečne vrednosti za oba modela.

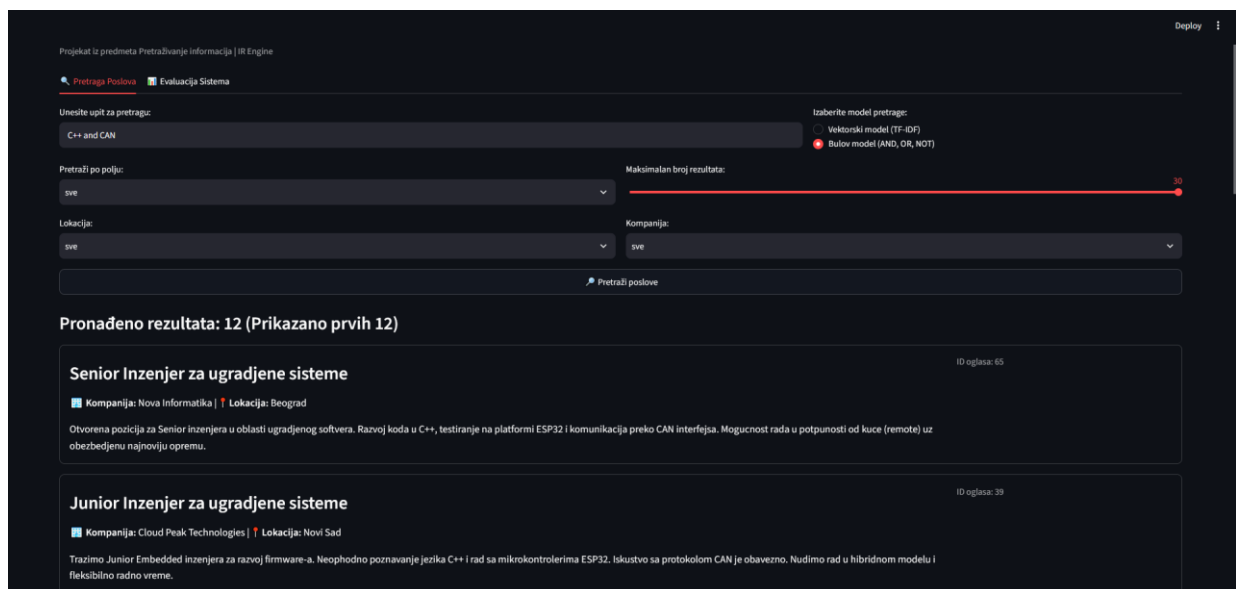
Kontrola	Namena
Upit	Tekst koji korisnik želi da pronađe
Model	Vektorski TF-IDF ili Bulov AND/OR/NOT
Polje	Sve, naslov ili opis
Lokacija	Tačan filter po lokaciji
Kompanija	Tačan filter po kompaniji
Top K	Maksimalan broj prikazanih rezultata

Tabela 3: Glavne kontrole Streamlit interfejsa.

Na slikama 4 i 5 prikazani su primeri rada aplikacije za dva različita modela pretrage. Prvi primer prikazuje vektorsku pretragu termina C++, dok drugi prikazuje Bulov upit C++ AND CAN.



Slika 4: Primer rezultata vektorske pretrage za upit C++.



Slika 5: Primer rezultata Bulove pretrage za upit C++ AND CAN.

Radi lakšeg tumačenja ponašanja sistema, Tabela 3a prikazuje nekoliko tipičnih upita i način na koji ih sistem interpretira.

Tabela 3a: Primeri upita i očekivano ponašanje sistema.

Primer upita	Model	Tumačenje / očekivani rezultat
Python Backend	Bulov	Tretira se kao Python AND Backend i vraća oglase koji sadrže oba termina.
C++	Vektorski	Rangira oglase po sličnošću i čuva termin C++ kao poseban tehnološki naziv.
C++ AND CAN	Bulov	Vrši presek skupova oglasa za termine C++ i CAN.
QA Inženjer	Vektorski	Sortira rezultate po TF-IDF sličnosti naslova i opisa sa upitom.

9. Rezultati i evaluacija sistema

Evaluacija se vrši nad prvih 100 oglasa iz korpusa. Za pet upita ručno je određen skup relevantnih dokumenata, što predstavlja ground truth. Korišćeni upiti su Python Backend, Frontend React, DevOps Docker, Java Spring i QA Inženjer. Relevantni skupovi sadrže redom 4, 1, 1, 3 i 5 oglasa.

Test upit	Ručno označeni relevantni ID-jevi
Python Backend	14, 22, 37, 58
Frontend React	49
DevOps Docker	30
Java Spring	31, 46, 78
QA Inženjer	2, 21, 24, 40, 75

Tabela 4: Ground truth korišćen u evaluaciji.

Precision pokazuje koji deo pronađenih dokumenata je relevantan, Recall pokazuje koji deo svih relevantnih dokumenata je pronađen, dok F1 predstavlja harmonijsku sredinu ove dve mere. Za vektorski model evaluira se prvih K = 10 rezultata.

Precision = tačno pronađeni / svi pronađeni

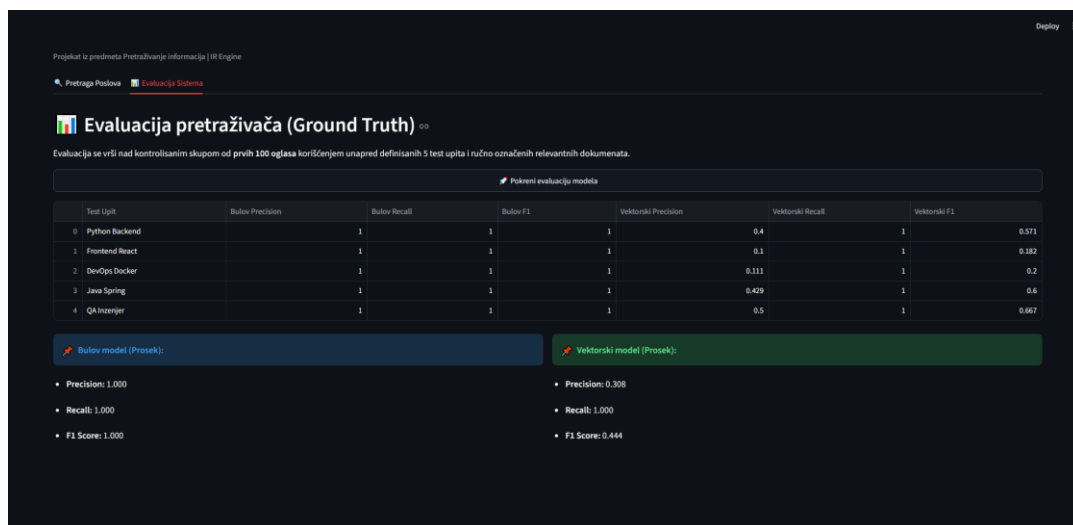
Recall = tačno pronađeni / svi relevantni

F1 = $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$

Upit	B P	B R	B F1	V P	V R	V F1
Python Backend	1.000	1.000	1.000	0.400	1.000	0.571
Frontend React	1.000	1.000	1.000	0.100	1.000	0.182
DevOps Docker	1.000	1.000	1.000	0.111	1.000	0.200
Java Spring	1.000	1.000	1.000	0.429	1.000	0.600
QA Inženjer	1.000	1.000	1.000	0.500	1.000	0.667
PROSEK	1.000	1.000	1.000	0.308	1.000	0.444

Tabela 5: Rezultati evaluacije Bulvog (B) i vektorskog (V) modela.

Na slici 6 prikazan je izgled kartice za evaluaciju sa tabelarnim prikazom Precision, Recall i F1 mera za oba modela.



Slika 6: Kartica za evaluaciju i prikaz rezultata nad ground truth skupom.

Za svih pet test upita Bulov model je postigao Precision, Recall i F1 jednako 1. Rezultat se odnosi na konkretni skup od pet test upita i prvih 100 oglasa, kod kojih se strogi AND princip poklopio sa ručno označenim relevantnim dokumentima.

Vektorski model ima Recall jednak 1 za svaki test upit, što znači da je u prvih 10 rezultata uspeo da pronađe sve ručno označene relevantne oglase. Precision je niži jer su uz relevantne dokumente vraćeni i dodatni kandidati sa pozitivnom TF-IDF sličnošću. Na primer, za Python Backend pronađena su sva četiri relevantna oglasa, ali je među prvih deset bilo i drugih oglasa, pa je Precision 0,4 i F1 0,571.

10. Prednosti, ograničenja i moguća unapređenja

Glavna prednost projekta je što u jednoj aplikaciji prikazuje dva različita pristupa pretraživanju. Bulov model daje preciznu kontrolu logičkih uslova, dok vektorski model omogućava rangiranje rezultata prema stepenu sličnosti. Preprocessing, indeksiranje, pretraga, filtriranje i evaluacija razdvojeni su u zasebne module, pa se tok podataka može lako pratiti.

Druga važna prednost je korišćenje polustrukturiranih dokumenata. Umesto da se naslov, kompanija, lokacija i opis spoje u jedan tekst, sistem razlikuje tekstualna polja od strukturiranih filtera. Ovim se izbegava da naziv kompanije ili grada nepotrebno utiče na TF-IDF skor, a korisnik i dalje može vrlo jednostavno da suzi skup rezultata.

Prilagođena tokenizacija razlikuje nazive C, C++, C#, .NET, ASP.NET, Node.js i Vue.js. Kod ovih termina specijalni znaci ostaju deo tokena, pa se ne primenjuje opšti postupak uklanjanja interpunkcije.

10.1. Ograničenja

- Bulov parser ne podržava zagrade i složenu sintaksu; prioritet je pojednostavljen na NOT/AND unutar OR grupa.
- Evaluacioni skup sadrži samo pet upita i prvih 100 oglasa, pa rezultati nisu dovoljni za opšti zaključak o kvalitetu modela.
- Korpus je sintetički generisan i jednostavniji je od stvarnih oglasa sa različitih sajtova.
- Vektorski model koristi klasični TF-IDF i ne razume sinonime ili semantički slične izraze koji ne dele iste termine.
- Ručno označeni relevantni ID-jevi vezani su za postojeću verziju jobs.json; ponovno nasumično generisanje korpusa zahtevalo bi novo označavanje ground truth-a.

10.2. Moguća unapređenja

- proširenje ground truth skupa na veći broj upita i dokumenata;
- podrška za zagrade u Bulovom parseru ako postane potrebna složena sintaksa;
- uvođenje BM25 rangiranja kao alternative TF-IDF modelu;
- dodavanje sinonima ili semantičkih reprezentacija za značenjski srodne termine;
- učitavanje stvarnih oglasa i dodavanje dodatnih strukturiranih atributa, kao što su tehnologije, senioritet ili tip rada.

11. Zaključak

U radu je opisan razvoj sistema za pretraživanje oglasa za posao koji povezuje osnovne principe pretraživanja informacija sa praktičnom Python implementacijom. Polustrukturirani JSON oglasi obrađuju se tako da naslov i opis učestvuju u tekstualnoj pretrazi, dok kompanija i lokacija ostaju strukturirani filteri. Time se čuva semantika pojedinačnih polja i omogućava fleksibilnije korišćenje podataka.

Implementirana su dva modela pretrage. Bulov model koristi invertovani indeks i operacije nad skupovima, podržava AND, OR i NOT i implicitni AND. Vektorski model koristi TF-IDF i kosinusnu sličnost, pri čemu naslov ima ponder 0,6, a opis 0,4 kada se pretražuju oba polja. Rezultati se korisniku prikazuju kroz jednostavan Streamlit interfejs sa izborom polja, filterima i ograničenjem broja rezultata.

Posebna pažnja posvećena je obradi IT termina sa specijalnim znacima kako bi se razlikovali, na primer, C, C++ i C#. Sistem je evaluiran nad ručno označenim ground truth skupom. U konkretnom testu Bulov model se potpuno poklopio sa relevantnim dokumentima, dok je vektorski model pronašao sve relevantne dokumente u prvih deset rezultata, ali uz nižu preciznost zbog dodatnih kandidata.

Sistem obuhvata automatsku obradu teksta, invertovani indeks, Bulov model, vektorsko rangiranje i evaluaciju pomoću Precision, Recall i F1 mera. Sve komponente su povezane kroz Streamlit interfejs za unos upita, filtriranje i prikaz rezultata.

12. Literatura

1. Materijali sa predavanja iz predmeta Pretraživanje informacija, Elektronski fakultet, Univerzitet u Nišu.
2. Manning, C. D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval, Cambridge University Press, 2008.
3. Scikit-learn dokumentacija: TfidfVectorizer i cosine_similarity.
4. Streamlit dokumentacija: razvoj interaktivnih Python aplikacija za prikaz podataka i rezultata.
5. Milošević, N.: Serbian stemmer, University of Manchester, School of Computer Science, 2015, GNU/GPL 3.0.
6. Python dokumentacija: skupovi (set) i osnovne skupovne operacije korišćene u Bulovom modelu.
7. NLTK dokumentacija: tokenizacija teksta i funkcija word_tokenize.